



BRAINWARE UNIVERSITY
School of Engineering
Department of Computer Science & Engineering – Cyber Security & Data Science
398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

COMPUTER ORGANIZATION AND ARCHITECTURE LABORATORY

EXPERIMENT NO. 10

Design and Verification of a 4 x 8-bit Register File Using ModelSim

1. Aim / Purpose of the Experiment

To design and simulate a simple 4 x 8-bit register file using Verilog HDL in ModelSim, and to verify synchronous write, reset, and simultaneous two-port read operations for different register addresses.

This experiment extends the single-register and Program Counter concepts of Experiments 8 and 9 to a small bank of CPU registers. It prepares students for memory-unit, ALU-datapath, and CPU-design experiments.

2. Learning Outcomes

- Explain the purpose of a register file in a processor datapath.
- Identify R0, R1, R2, and R3 as four independent 8-bit storage registers.
- Use a 2-bit address to select one of four registers.
- Implement a synchronous single write port controlled by WRITE_ENABLE.
- Implement two combinational read ports so that two registers can be observed at the same time.
- Use Verilog case statements for register selection.
- Interpret ModelSim waveforms for reset, write, and read operations.
- Relate two-read-port register files to the two operands required by an ALU.

3. Prerequisites

- Binary and hexadecimal representation of 8-bit data.
- Basic 8-bit register operation and positive-edge-triggered storage.
- Verilog module, always block, if-else, case statement, and non-blocking assignment (<=).
- Basic ModelSim project creation, compilation, simulation, and waveform observation.

4. Materials / Equipment / Apparatus / Devices / Software Required

- Computer or laptop with Windows or Linux operating system.
- ModelSim - Intel FPGA Edition, ModelSim PE, or QuestaSim installed and configured.
- Text editor integrated with ModelSim or an external HDL editor.
- Verilog source files: register_file_4x8.v and tb_register_file_4x8.v.
- Laboratory record notebook for register contents, read-port observations, waveform interpretation, and result.



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science & Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

5. Introduction and Theory

A register file is a small collection of high-speed registers organised so that one or more registers can be selected by address. Register files are widely used inside CPUs to hold operands, temporary values, addresses, and intermediate ALU results. Unlike a single register, a register file uses address inputs to select which storage location will be written or read.

5.1 Organisation of the 4 x 8-bit Register File

The proposed register file contains four registers named R0, R1, R2, and R3. Each register stores one 8-bit word. Therefore, the total storage capacity is $4 \times 8 = 32$ bits. A 2-bit address is sufficient to select one of the four registers because $2^2 = 4$.

Address	Selected Register	Width
00	R0	8 bits
01	R1	8 bits
10	R2	8 bits
11	R3	8 bits

The design uses one synchronous write port and two combinational read ports. The write address chooses the register that receives WRITE_DATA at the next positive clock edge when WRITE_ENABLE is 1. READ_ADDR1 and READ_ADDR2 independently select the values presented on READ_DATA1 and READ_DATA2.

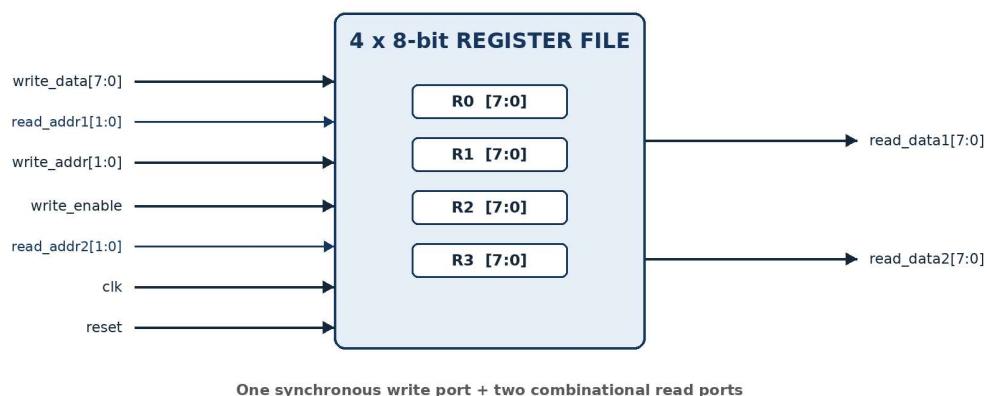


Figure 1. Functional block diagram of the 4 x 8-bit register file.

5.2 Write Operation

Writing is synchronous. Therefore, a register changes only at the positive edge of CLK. If RESET = 1, all registers are cleared. Otherwise, when WRITE_ENABLE = 1, WRITE_ADDR selects exactly one register and WRITE_DATA is stored in that register.



BRAINWARE UNIVERSITY
School of Engineering
Department of Computer Science & Engineering – Cyber Security & Data Science
398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

RESET	WRITE_ENABLE	WRITE_ADDR	Operation
1	X	XX	R0, R1, R2, R3 <- 00H
0	0	XX	No register is modified
0	1	00	R0 <- WRITE_DATA
0	1	01	R1 <- WRITE_DATA
0	1	10	R2 <- WRITE_DATA
0	1	11	R3 <- WRITE_DATA

Important: WRITE is clock-controlled. The selected register is updated only at a positive clock edge.

5.3 Dual Read Operation

The two read ports are combinational. Changing READ_ADDR1 or READ_ADDR2 immediately changes the corresponding read output; no clock edge is required. This allows two stored values to be supplied simultaneously to a later ALU experiment.

Example: if R0 = AAH, R1 = 55H, R2 = F0H, and R3 = 0FH, then READ_ADDR1 = 00 and READ_ADDR2 = 10 produce READ_DATA1 = AAH and READ_DATA2 = F0H at the same time.

5.4 Why Two Read Ports Are Useful in a CPU

Many arithmetic and logical operations require two source operands. A dual-read register file can provide Operand A and Operand B simultaneously, while the write port can later store an ALU result into a selected destination register. This is a simplified version of the organisation used in processor datapaths.

Key idea: WRITE = clock controlled; READ = address controlled.

5.5 ModelSim Design and Verification Flow

Stage	Activity	Purpose
1	Create project and work library	Organise HDL source and testbench files.
2	Write register_file_4x8 module	Implement reset, one write port, and two read ports.
3	Write testbench	Apply simple write and read sequences.



BRAINWARE UNIVERSITY
School of Engineering
Department of Computer Science & Engineering – Cyber Security & Data Science
398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

Stage	Activity	Purpose
4	Compile source files	Detect syntax and elaboration errors.
5	Run simulation	Observe register contents and read outputs.
6	Verify results	Compare ModelSim values with the expected table.

6. Operating Procedure

Part A: Create the ModelSim Project

1. Open ModelSim and select File -> New -> Project.
2. Enter the project name as COA_Experiment_10 and choose a suitable project directory.
3. Keep the default work library and create the project.
4. Create two Verilog files: register_file_4x8.v and tb_register_file_4x8.v.
5. Save both files in the same project directory.

Part B: Write the 4 x 8-bit Register File

Enter the following beginner-friendly synthesizable Verilog program in register_file_4x8.v. The registers are declared separately so that students can easily see R0, R1, R2, and R3 and understand which address selects each register.



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science & Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

VERILOG DESIGN MODULE - register_file_4x8.v

```
01 // File: register_file_4x8.v
02 // Experiment 10 - 4 x 8-bit Register File
03
04 module register_file_4x8 (
05     input clk,
06     input reset,
07     input write_enable,
08     input [1:0] write_addr,
09     input [1:0] read_addr1,
10     input [1:0] read_addr2,
11     input [7:0] write_data,
12     output reg [7:0] read_data1,
13     output reg [7:0] read_data2
14 );
15
16 // Four 8-bit registers
17 reg [7:0] R0;
18 reg [7:0] R1;
19 reg [7:0] R2;
20 reg [7:0] R3;
21
22 // WRITE OPERATION
23 always @(posedge clk)
24 begin
25     if (reset)
26     begin
27         R0 <= 8'b00000000;
28         R1 <= 8'b00000000;
29         R2 <= 8'b00000000;
30         R3 <= 8'b00000000;
31     end
32     else if (write_enable)
33     begin
34         case (write_addr)
35             2'b00: R0 <= write_data;
36             2'b01: R1 <= write_data;
37             2'b10: R2 <= write_data;
38             2'b11: R3 <= write_data;
39         endcase
40     end
41 end
42
43 // READ PORT 1
44 always @(*)
45 begin
46     case (read_addr1)
47         2'b00: read_data1 = R0;
48         2'b01: read_data1 = R1;
49         2'b10: read_data1 = R2;
50         2'b11: read_data1 = R3;
51     endcase
52 end
53
54 // READ PORT 2
55 always @(*)
56 begin
57     case (read_addr2)
58         2'b00: read_data2 = R0;
59         2'b01: read_data2 = R1;
60         2'b10: read_data2 = R2;
61         2'b11: read_data2 = R3;
62     endcase
63 end
64
65 endmodule
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science & Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

Part B.1: Easy Statement-by-Statement Explanation

Verilog statement	Meaning for students
module register_file_4x8 (...)	Declares the complete register-file hardware block.
input [1:0] write_addr	Two address bits select R0, R1, R2, or R3 for writing.
input [1:0] read_addr1 / read_addr2	Two independent addresses allow two registers to be read at the same time.
reg [7:0] R0 ... R3	Creates four separate 8-bit storage registers.
always @(posedge clk)	The write section operates only on the rising edge of the clock.
if (reset)	RESET has highest priority and clears all four registers.
else if (write_enable)	Writing is allowed only when WRITE_ENABLE = 1.
case (write_addr)	Selects which register receives WRITE_DATA.
R2 <= write_data	Example: address 10 stores WRITE_DATA in R2.
always @(*)	Describes combinational read logic; no clock is needed.
case (read_addr1)	Selects the value appearing on READ_DATA1.
case (read_addr2)	Selects the value appearing on READ_DATA2.

Part C: Write the Testbench

The following simple testbench writes four different hexadecimal values into R0-R3 and then reads two registers at a time. The sequence is deliberately short so that students can easily verify it in the ModelSim Wave window.

VERILOG TESTBENCH - tb_register_file_4x8.v

```
01 `timescale 1ns/1ps
02
03 module tb_register_file_4x8;
04
05 reg clk;
06 reg reset;
07 reg write_enable;
08 reg [1:0] write_addr;
09 reg [1:0] read_addr1;
10 reg [1:0] read_addr2;
11 reg [7:0] write_data;
12
13 wire [7:0] read_data1;
14 wire [7:0] read_data2;
15
16 register_file_4x8 uut (
17     .clk(clk),
18     .reset(reset),
19     .write_enable(write_enable),
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science & Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

```
20     .write_addr(write_addr),
21     .read_addr1(read_addr1),
22     .read_addr2(read_addr2),
23     .write_data(write_data),
24     .read_data1(read_data1),
25     .read_data2(read_data2)
26 );
27
28 // Clock: 10 ns period
29 always #5 clk = ~clk;
30
31 initial
32 begin
33     clk = 0;
34     reset = 1;
35     write_enable = 0;
36     write_addr = 2'b00;
37     read_addr1 = 2'b00;
38     read_addr2 = 2'b00;
39     write_data = 8'h00;
40
41     // Reset all registers
42     #10;
43     reset = 0;
44
45     // Write AAH into R0
46     #10;
47     write_enable = 1;
48     write_addr = 2'b00;
49     write_data = 8'hAA;
50
51     // Write 55H into R1
52     #10;
53     write_addr = 2'b01;
54     write_data = 8'h55;
55
56     // Write F0H into R2
57     #10;
58     write_addr = 2'b10;
59     write_data = 8'hF0;
60
61     // Write 0FH into R3
62     #10;
63     write_addr = 2'b11;
64     write_data = 8'h0F;
65
66     // Stop writing
67     #10;
68     write_enable = 0;
69
70     // Read R0 and R1
71     read_addr1 = 2'b00;
72     read_addr2 = 2'b01;
73     #10;
74
75     // Read R2 and R3
76     read_addr1 = 2'b10;
77     read_addr2 = 2'b11;
78     #20;
79
80     $stop;
81 end
82
83 endmodule
```



BRAINWARE UNIVERSITY
School of Engineering
Department of Computer Science & Engineering – Cyber Security & Data Science
398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

Part C.1: Testbench Sequence Explained

Step	Stimulus	Expected effect
1	RESET = 1	R0-R3 become 00H at the next positive edge.
2	WRITE_ADDR=00, WRITE_DATA=AAH	R0 stores AAH.
3	WRITE_ADDR=01, WRITE_DATA=55H	R1 stores 55H.
4	WRITE_ADDR=10, WRITE_DATA=F0H	R2 stores F0H.
5	WRITE_ADDR=11, WRITE_DATA=0FH	R3 stores 0FH.
6	READ_ADDR1=00, READ_ADDR2=01	READ_DATA1=AAH and READ_DATA2=55H.
7	READ_ADDR1=10, READ_ADDR2=11	READ_DATA1=F0H and READ_DATA2=0FH.

Part D: Compile and Simulate

1. Select Compile -> Compile All and confirm that both Verilog files compile without errors.
2. Select Simulate -> Start Simulation and choose tb_register_file_4x8 from the work library.
3. Add clk, reset, write_enable, write_addr, write_data, read_addr1, read_addr2, read_data1, read_data2, uut.R0, uut.R1, uut.R2, and uut.R3 to the Wave window.
4. Display write_data, read_data1, read_data2, R0, R1, R2, and R3 in hexadecimal radix.
5. Run the simulation using Run -All and select Zoom Full.
6. Verify the four stored register values and the two dual-read cases shown in the expected-results tables.

Part E: Equivalent ModelSim Transcript Commands

MODELSIM TRANSCRIPT COMMANDS

```
01 vlib work
02 vlog register_file_4x8.v
03 vlog tb_register_file_4x8.v
04 vsim work.tb_register_file_4x8
05 add wave *
06 add wave sim:/tb_register_file_4x8/uut/R0
07 add wave sim:/tb_register_file_4x8/uut/R1
08 add wave sim:/tb_register_file_4x8/uut/R2
09 add wave sim:/tb_register_file_4x8/uut/R3
10 run -all
11 wave zoom full
```



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science & Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

8. Observation and Record

8.1 Expected Register Contents After Writing

Register	Address	Stored value (Hex)	Stored value (Binary)
R0	00	AAH	10101010
R1	01	55H	01010101
R2	10	F0H	11110000
R3	11	0FH	00001111

8.2 Expected Dual-Read Results

READ_ADDR1	READ_ADDR2	READ_DATA1	READ_DATA2
00	01	AAH	55H
10	11	F0H	0FH

8.3 Representative Expected Waveform

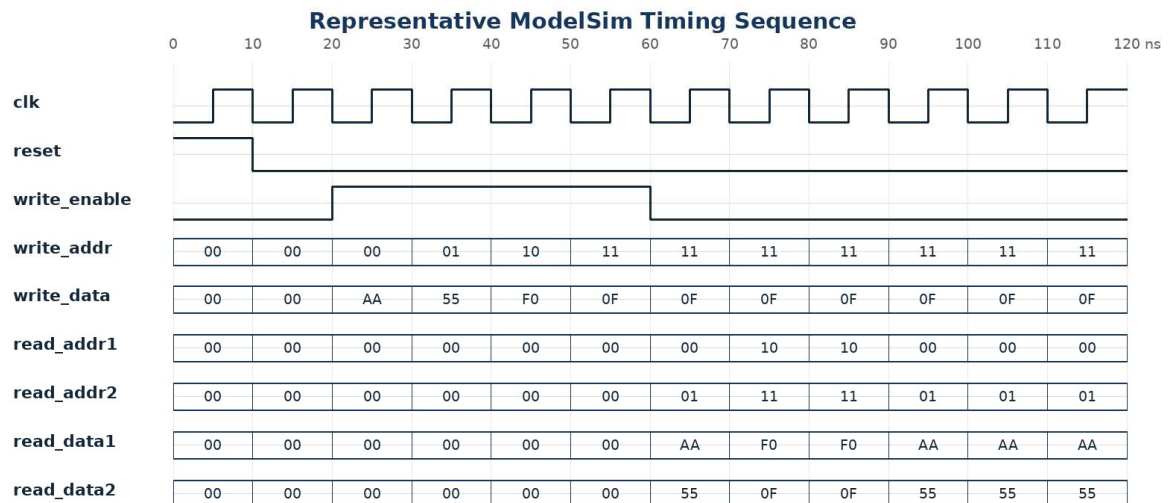


Figure 2. Representative timing sequence for reset, writing R0-R3, and dual-read verification.

10. Result / Conclusion

A 4 x 8-bit register file was designed using simple Verilog HDL and simulated in ModelSim. The reset operation correctly cleared all four registers. With WRITE_ENABLE active, AAH, 55H, F0H, and 0FH were stored in R0, R1, R2, and R3 respectively at positive clock edges. The two combinational read



BRAINWARE UNIVERSITY

School of Engineering

Department of Computer Science & Engineering – Cyber Security & Data Science

398, Ramkrishnapur Road, Barasat, North 24 Parganas, Kolkata - 700 125

ports successfully produced two selected register values simultaneously. Thus, synchronous writing and dual asynchronous reading of the register file were verified.

11. Viva-Voce Questions

1. What is a register file?
2. How many bits can a 4 x 8-bit register file store?
3. Why are two address bits enough for four registers?
4. What is the purpose of WRITE_ENABLE?
5. When does a write operation occur in this experiment?

12. Assessment Scheme

Assessment Component	Suggested Weightage
Circuit understanding and pre-lab preparation	10%
Correct Verilog design module	25%
Correct ModelSim testbench	20%
Waveform and result verification	20%
Observation / analysis record	10%
Viva voce	15%

13. Suggested Readings

- M. Morris Mano and Michael D. Ciletti, Digital Design: With an Introduction to the Verilog HDL, Pearson.
- M. Morris Mano, Computer System Architecture, Pearson.
- William Stallings, Computer Organization and Architecture, Pearson.
- Samuel P. Harbison and Guy L. Steele Jr., Verilog HDL references and ModelSim/QuestaSim user documentation for simulation commands.